PROGRAM

```
import csv
import random
import math
```

Load CSV file

```
def loadcsv(filename):
    lines = csv.reader(open(filename, "r"))
    # Skip the header row
    next(lines)
    dataset = list(lines)
    for i in range(len(dataset)):
        # Convert strings into numbers
        dataset[i] = [float(x) for x in dataset[i]]
    return dataset
```

Split dataset into training and testing data

```
def splitdataset(dataset, splitratio):
    # 67% training size
    trainsize = int(len(dataset) * splitratio)
    trainset = []
    copy = list(dataset)
    while len(trainset) < trainsize:
        # Generate random index
        index = random.randrange(len(copy))
        trainset.append(copy.pop(index))
    return [trainset, copy]
```

Separate dataset according to class

```
def separatebyclass(dataset):
    separated = {}
    # Separate data according to class
    for i in range(len(dataset)):
        vector = dataset[i]
        if vector[-1] not in separated:
            separated[vector[-1]] = []
        separated[vector[-1]].append(vector)
    return separated
```

Calculate mean

```
def mean(numbers):
    return sum(numbers) / float(len(numbers))
```

Calculate standard deviation

```
def stdev(numbers):
    avg = mean(numbers)
    variance = sum(
        [pow(x - avg, 2) for x in numbers]
    ) / float(len(numbers) - 1)
    return math.sqrt(variance)
```

Calculate mean and standard deviation for each attribute

```
def summarize(dataset):
    summaries = [
        (mean(attribute), stdev(attribute))
        for attribute in zip(*dataset)
    ]
    del summaries[-1]
    return summaries
```

Calculate summaries for each class

```
def summarizebyclass(dataset):
    separated = separatebyclass(dataset)
    summaries = {}
    for classvalue, instances in separated.items():
        # Calculate mean and standard deviation
        summaries[classvalue] = summarize(instances)
    return summaries

# Calculate Gaussian probability
def calculateprobability(x, mean, stdev):
    exponent = math.exp(
        -(math.pow(x - mean, 2) /
          (2 * math.pow(stdev, 2)))
    )
    return (
        (1 / (math.sqrt(2 * math.pi) * stdev))
        * exponent
    )
```

Calculate probability for each class

```
def calculateclassprobabilities(summaries, inputvector):
    probabilities = {}
    for classvalue, classsummaries in summaries.items():
        probabilities[classvalue] = 1
        for i in range(len(classsummaries)):
```

```
            mean_value, stdev_value = classsummaries[i]
            x = inputvector[i]
            probabilities[classvalue] *= calculateprobability(
                x,
                mean_value,
                stdev_value
            )
    return probabilities
```

Predict class

```
def predict(summaries, inputvector):
    probabilities = calculateclassprobabilities(
        summaries,
        inputvector
    )

    bestLabel = None
    bestProb = -1
    for classvalue, probability in probabilities.items():
        if bestLabel is None or probability > bestProb:
            bestProb = probability
            bestLabel = classvalue
    return bestLabel
```

Get predictions for test data

```
def getpredictions(summaries, testset):
    predictions = []
    for i in range(len(testset)):
        result = predict(
            summaries,
            testset[i]
        )
        predictions.append(result)
    return predictions
```

Calculate accuracy

```
def getaccuracy(testset, predictions):
    correct = 0
    for i in range(len(testset)):
        if testset[i][-1] == predictions[i]:
            correct += 1
    return (correct / float(len(testset))) * 100.0
```

Main function

```
def main():
    # CSV file name
    filename = '/content/Naive-Bayes-Classification-Data.csv'
    splitratio = 0.67
    dataset = loadcsv(filename)
```

Split dataset

```
    trainingset, testset = splitdataset(
        dataset,
        splitratio
    )
    print(
        'Split {0} rows into train={1} and test={2} rows'
        .format(
            len(dataset),
            len(trainingset),
            len(testset)
        )
    )
```

Prepare model

```
    summaries = summarizebyclass(trainingset)
```

Predict test data

```
    predictions = getpredictions(
        summaries,
        testset
    )
    accuracy = getaccuracy(
        testset,
        predictions
    )
print(
        'Accuracy of the classifier is : {0}%'
        .format(accuracy)
    )
```

Run the program

```
if __name__ == "__main__":
    main()
```

OUTPUT

```
Split 995 rows into train=666 and test=329 rows
Accuracy of the classifier is : 92.09726443768997%
```